

Question 1: Student Subject Averages using NumPy

```
import numpy as np

student_scores = np.array([
    [80, 75, 85, 70],
    [90, 85, 80, 75],
    [70, 80, 90, 85],
    [85, 90, 95, 80]
])

subjects = ["Math", "Science", "English", "History"]

average = np.mean(student_scores, axis=0)

for i in range(4):
    print(subjects[i], "Average =", average[i])

highest = np.argmax(average)
print("Highest average subject:", subjects[highest])
```

... Math Average = 81.25
Science Average = 82.5
English Average = 87.5
History Average = 77.5
Highest average subject: English

Question 2: Average Product Price using NumPy

```
import numpy as np

prices = np.array([
    [100, 120, 150],
    [80, 90, 110],
    [200, 180, 160]
])

average_price = np.mean(prices)

print("Average price =", average_price)
```

Average price = 132.22222222222223

Question 3: Average Sale Price of Houses with More Than 4 Bedrooms

```
import numpy as np

# Columns: Bedrooms, Area, Sale Price
house_data = np.array([
    [3, 1200, 250000],
    [5, 1800, 400000],
    [6, 2200, 500000],
    [4, 1500, 320000],
    [5, 2000, 450000]
])

selected = house_data[house_data[:, 0] > 4]
average_price = np.mean(selected[:, 2])

print("Average sale price =", average_price)
```

Average sale price = 450000.0

Question 4: Total Yearly Sales and Percentage Increase

```
 import numpy as np

sales_data = np.array([10000, 12000, 15000, 18000])

total_sales = np.sum(sales_data)
percentage_increase = ((sales_data[3] - sales_data[0])
                       / sales_data[0]) * 100

print("Total sales =", total_sales)
print("Percentage increase =", percentage_increase, "%")
```

```
... Total sales = 55000
    Percentage increase = 80.0 %
```

Question 5: Average Fuel Efficiency and Percentage Improvement

```
import numpy as np

fuel_efficiency = np.array([25, 30, 28, 35, 32])

average = np.mean(fuel_efficiency)

old_model = fuel_efficiency[0]
new_model = fuel_efficiency[1]

improvement = ((new_model - old_model) / old_model) * 100

print("Average fuel efficiency =", average)
print("Percentage improvement =", improvement, "%")
```

```
... Average fuel efficiency = 30.0
    Percentage improvement = 20.0 %
```

EX:6: Total Grocery Cost with Discount and Tax

PROGRAM CODE:

```
prices = [100, 50, 200]
quantities = [2, 3, 1]

discount_rate = 10
tax_rate = 5

subtotal = 0

for price, quantity in zip(prices, quantities):
    subtotal += price * quantity

discount = subtotal * discount_rate / 100
after_discount = subtotal - discount

tax = after_discount * tax_rate / 100
total = after_discount + tax

print("Subtotal =", subtotal)
print("Discount =", discount)
print("Tax =", tax)
print("Total cost =", total)
```

OUTPUT:

```
... Subtotal = 550
    Discount = 55.0
    Tax = 24.75
    Total cost = 519.75
```

Question 7: Pandas Order Analysis

PROGRAM CODE:

```
import pandas as pd

order_data = pd.DataFrame({
    "Customer_ID": [1, 1, 2, 2, 3],
    "Order_Date": ["2026-01-02", "2026-01-05", "2026-01-03",
                  "2026-01-10", "2026-01-08"],
    "Product": ["Pen", "Book", "Pen", "Bag", "Book"],
    "Quantity": [2, 3, 5, 1, 4]
})

print("Orders by customer:")
print(order_data.groupby("Customer_ID").size())

print("\nAverage quantity by product:")
print(order_data.groupby("Product")["Quantity"].mean())

order_data["Order_Date"] = pd.to_datetime(order_data["Order_Date"])

print("\nEarliest date:", order_data["Order_Date"].min())
print("Latest date:", order_data["Order_Date"].max())
```

OUTPUT:

```
*** Orders by customer:
Customer_ID
1      2
2      2
3      1
dtype: int64

Average quantity by product:
Product
Bag      1.0
Book     3.5
Pen      3.5
Name: Quantity, dtype: float64

Earliest date: 2026-01-02 00:00:00
Latest date: 2026-01-10 00:00:00
```

Question 8: Top 5 Products Sold

```
import pandas as pd

sales = pd.DataFrame({
    "Product": ["Pen", "Book", "Pen", "Bag", "Book", "Pen", "Bag"],
    "Quantity": [10, 8, 15, 5, 12, 20, 9]
})

top5 = sales.groupby("Product")["Quantity"].sum()
top5 = top5.sort_values(ascending=False).head(5)

print(top5)
```

```
*** Product
Pen      45
Book     20
Bag      14
Name: Quantity, dtype: int64
```

Question 9: Property Data Analysis using Pandas

```
import pandas as pd
```

```
property_data = pd.DataFrame({  
    "Location": ["Chennai", "Chennai", "Madurai", "Madurai", "Chennai"],  
    "Bedrooms": [3, 5, 4, 6, 5],  
    "Area": [1200, 1800, 1500, 2200, 2000],  
    "Price": [300000, 450000, 350000, 500000, 480000]  
})
```

```
print("Average price by location:")  
print(property_data.groupby("Location")["Price"].mean())
```

```
print("Properties with more than 4 bedrooms:")  
print((property_data["Bedrooms"] > 4).sum())
```

```
largest = property_data.loc[property_data["Area"].idxmax()]  
print("Property with largest area:")  
print(largest)
```

OUTPUT:

```
... Average price by location:  
Location  
Chennai    410000.0  
Madurai    425000.0  
Name: Price, dtype: float64  
Properties with more than 4 bedrooms:  
3  
Property with largest area:  
Location    Madurai  
Bedrooms      6  
Area         2200  
Price        500000  
Name: 3, dtype: object
```

Question 10: Monthly Sales – Line Plot and Bar Plot

PROGRAM CODE:

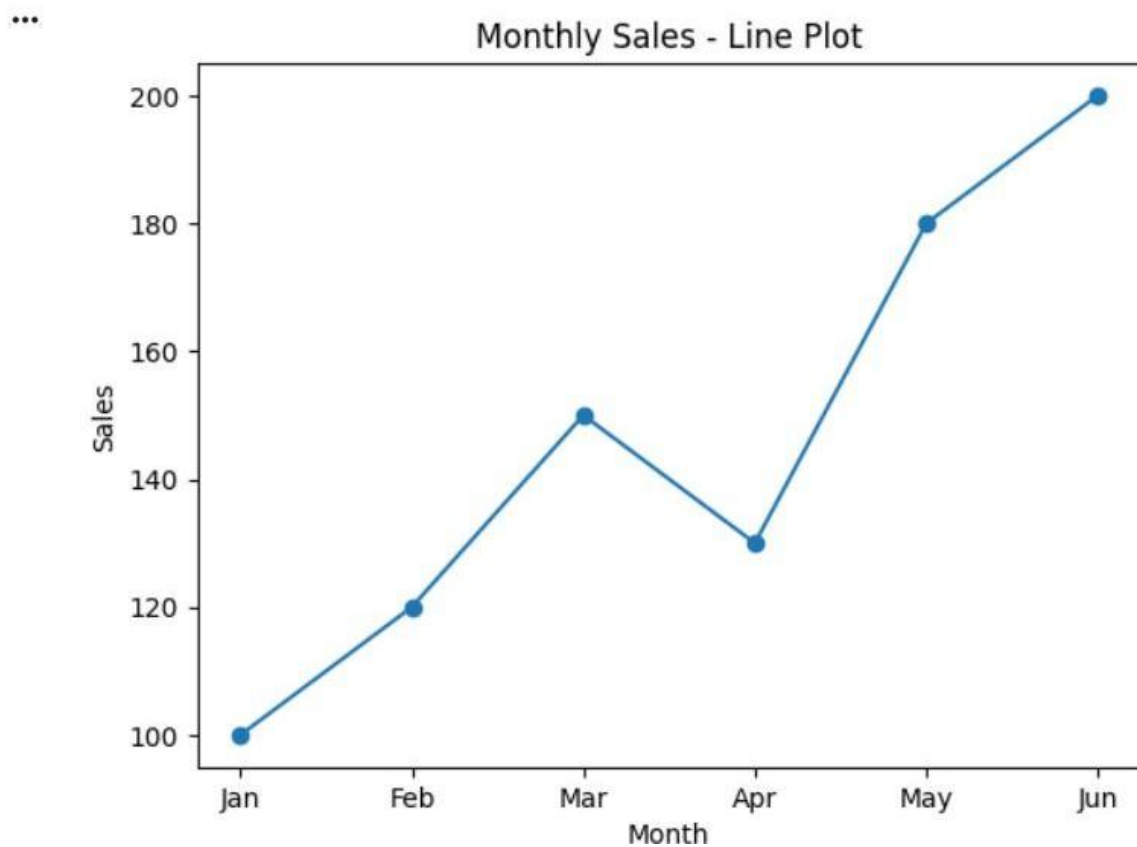
```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [100, 120, 150, 130, 180, 200]

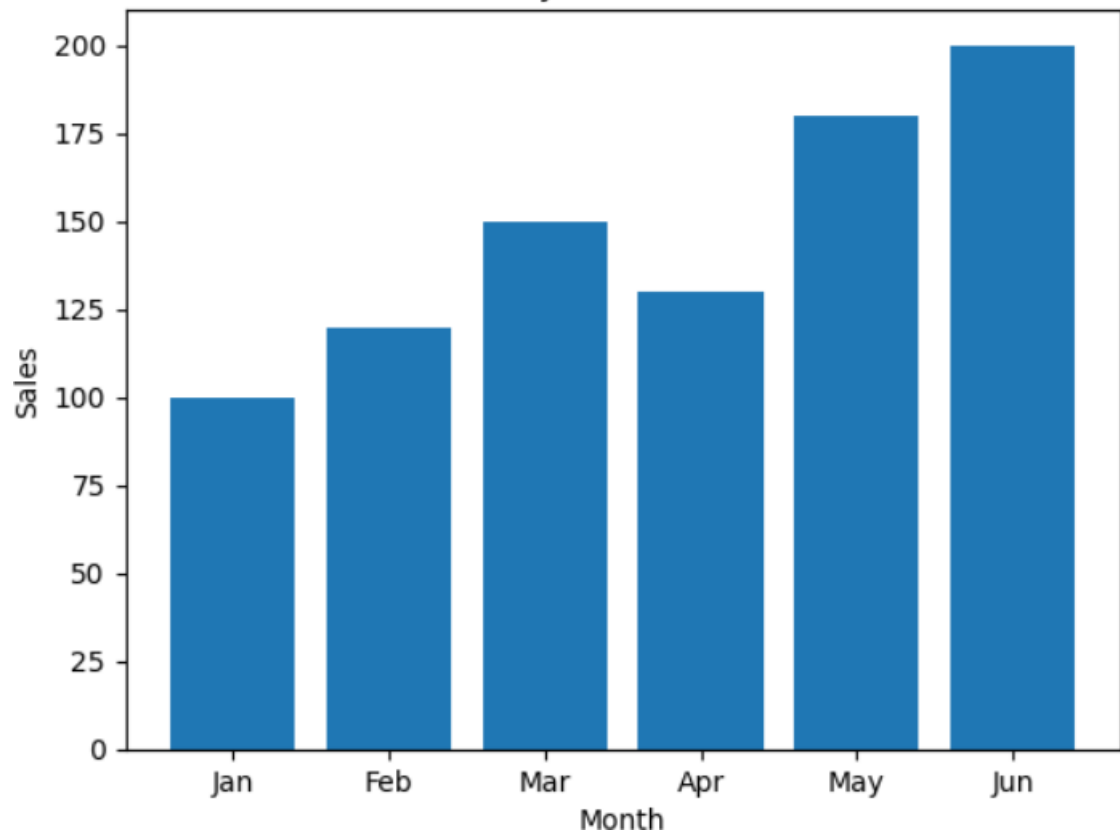
plt.plot(months, sales, marker="o")
plt.title("Monthly Sales - Line Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

plt.bar(months, sales)
plt.title("Monthly Sales - Bar Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```

OUTPUT:



Monthly Sales - Bar Plot



Question 11: Sales over Time – Line, Scatter and Bar Plots

```
import matplotlib.pyplot as plt
```

```
months = [1, 2, 3, 4, 5, 6]
```

```
sales = [100, 120, 150, 140, 180, 200]
```

```
plt.plot(months, sales, marker="o")
```

```
plt.title("Sales over Time")
```

```
plt.xlabel("Month")
```

```
plt.ylabel("Sales")
```

```
plt.show()
```

```
plt.scatter(months, sales)
```

```
plt.title("Sales - Scatter Plot")
```

```
plt.xlabel("Month")
```

```
plt.ylabel("Sales")
```

```
plt.show()
```

```
plt.bar(months, sales)
```

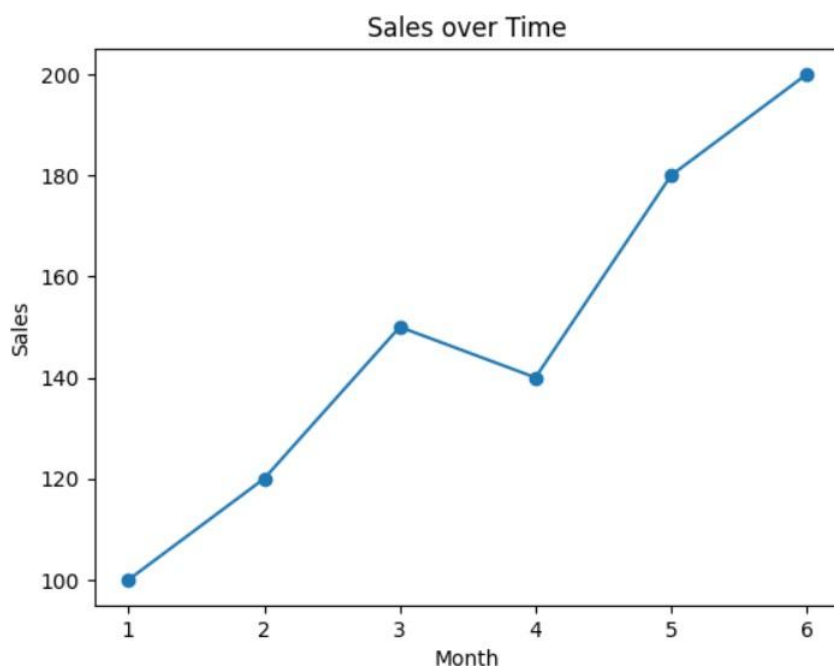
```
plt.title("Monthly Sales - Bar Plot")
```

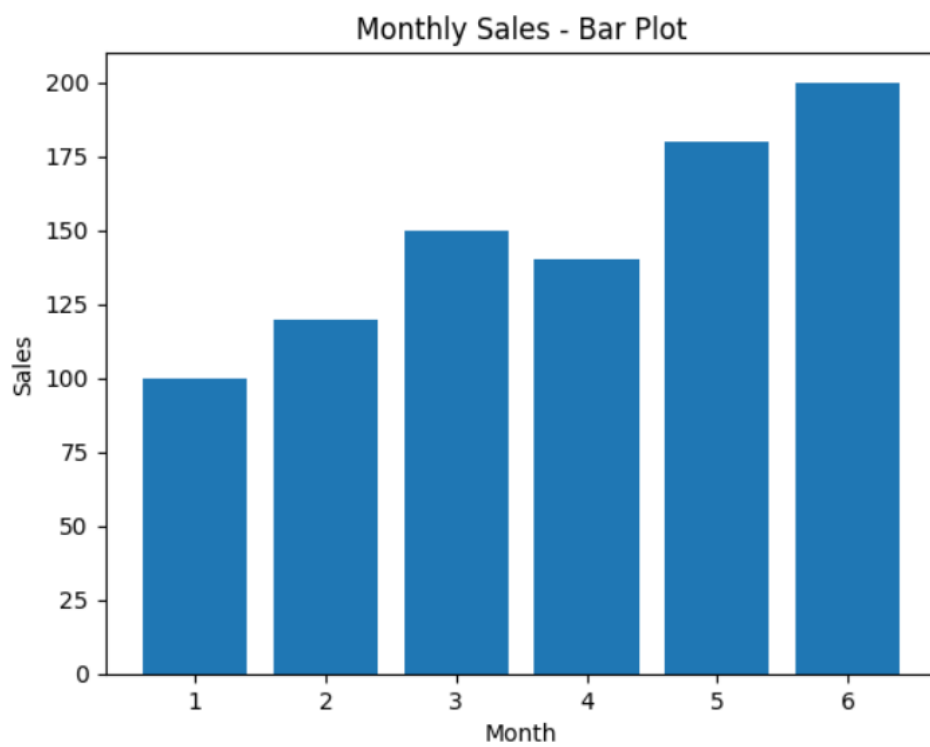
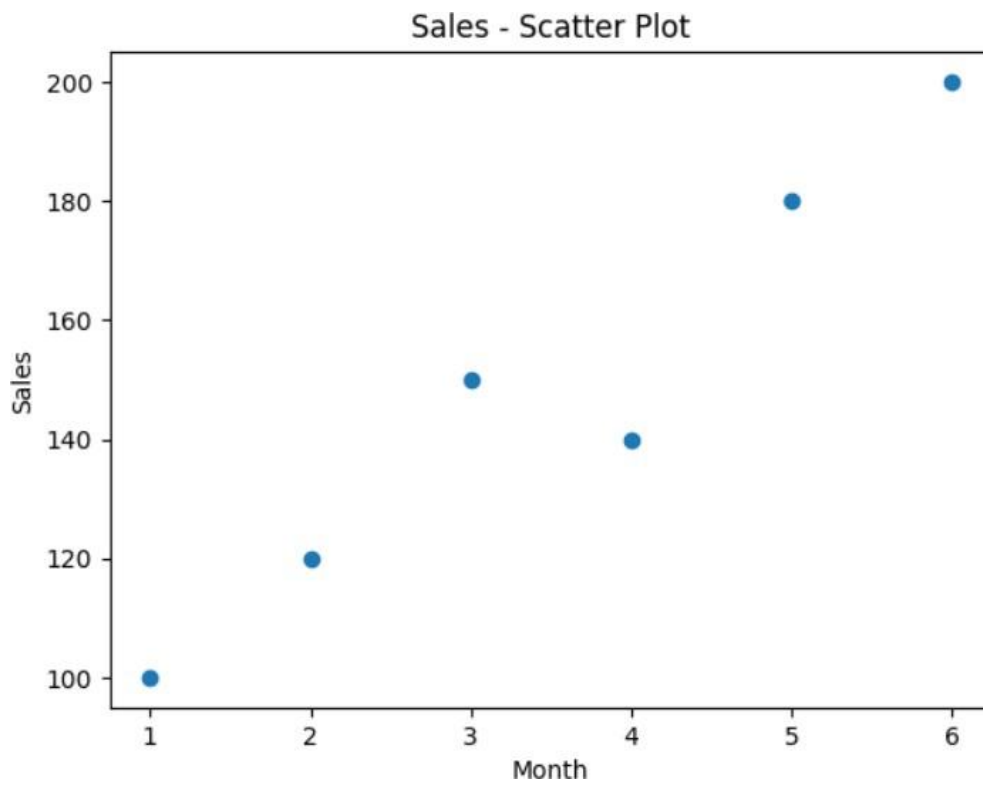
```
plt.xlabel("Month")
```

```
plt.ylabel("Sales")
```

```
plt.show()
```

OUTPUT:





Question 12: Monthly Temperature and Rainfall Plots

```
import matplotlib.pyplot as plt
```

```
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
```

```
temperature = [25, 27, 29, 31, 32, 30]
```

```
rainfall = [20, 15, 30, 80, 120, 100]
```

```
plt.plot(months, temperature, marker="o")
```

```
plt.title("Monthly Temperature")
```

```
plt.xlabel("Month")
```

```
plt.ylabel("Temperature")
```

```
plt.show()
```

```
plt.scatter(months, rainfall)
```

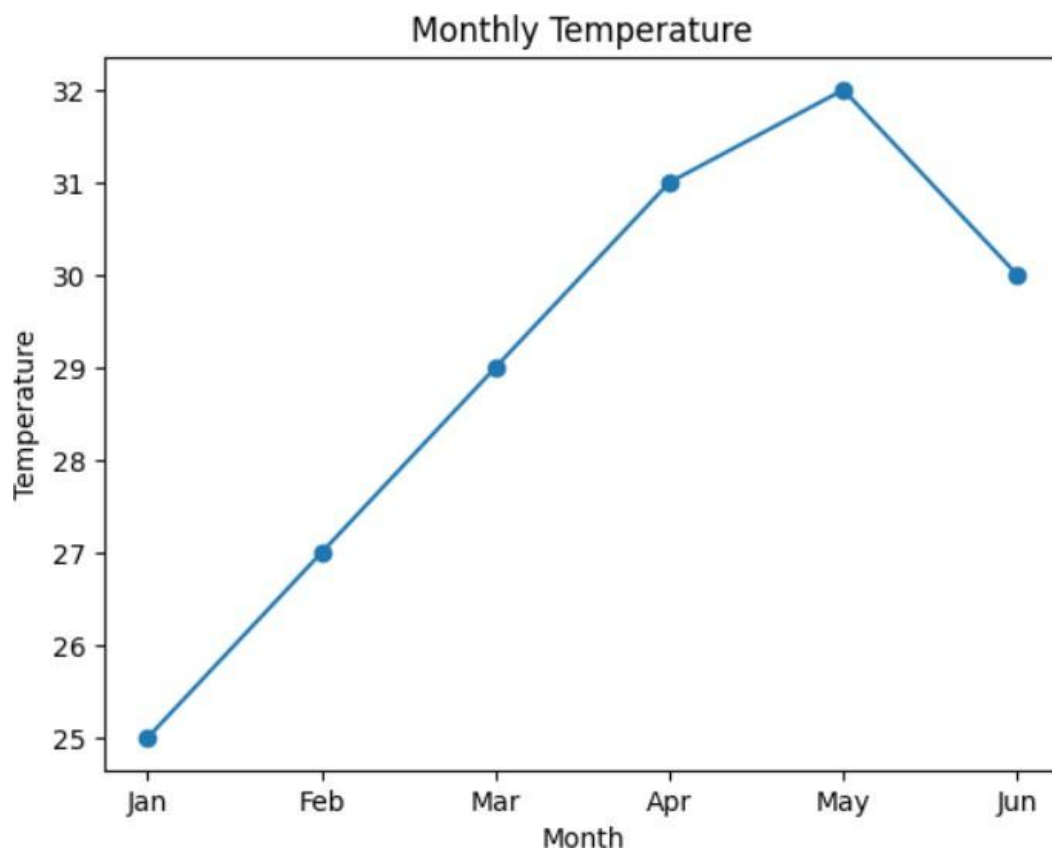
```
plt.title("Monthly Rainfall")
```

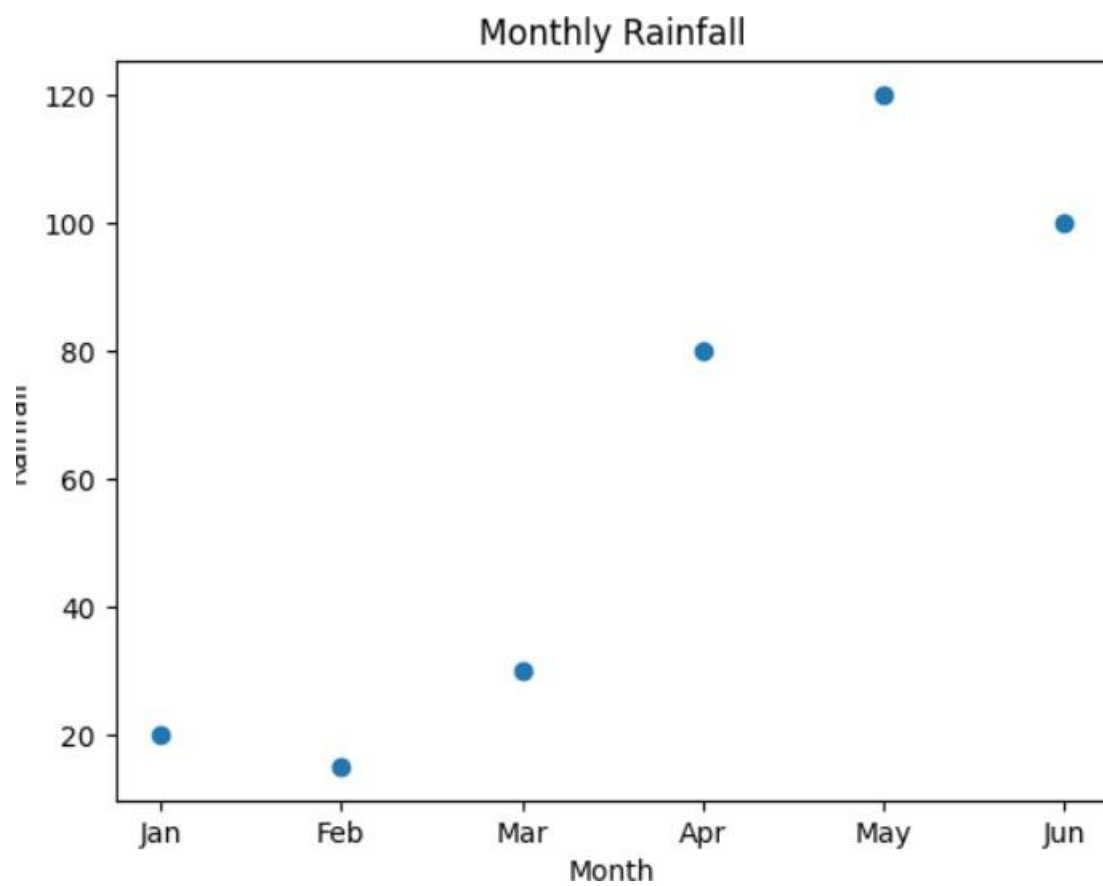
```
plt.xlabel("Month")
```

```
plt.ylabel("Rainfall")
```

```
plt.show()
```

OUTPUT:





Question 13: Word Frequency from a Text File

```
from collections import Counter

import re

# Create sample text file

with open("sample_text.txt", "w") as file:

    file.write("Python is easy to learn. Python is powerful. Python is popular.")

# Read the file

with open("sample_text.txt", "r") as file:

    text = file.read().lower()

# Extract words

words = re.findall(r"\b\w+\b", text)

# Count word frequency

frequency = Counter(words)

# Display result

for word, count in frequency.most_common():

    print(word, ":", count)
```

OUTPUT:

```
python : 3
is : 3
easy : 1
to : 1
learn : 1
powerful : 1
popular : 1
```

Question 14: Frequency Distribution of Customer Ages

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.DataFrame({
    "Age": [20, 25, 20, 30, 25, 20, 35, 30, 25]
})

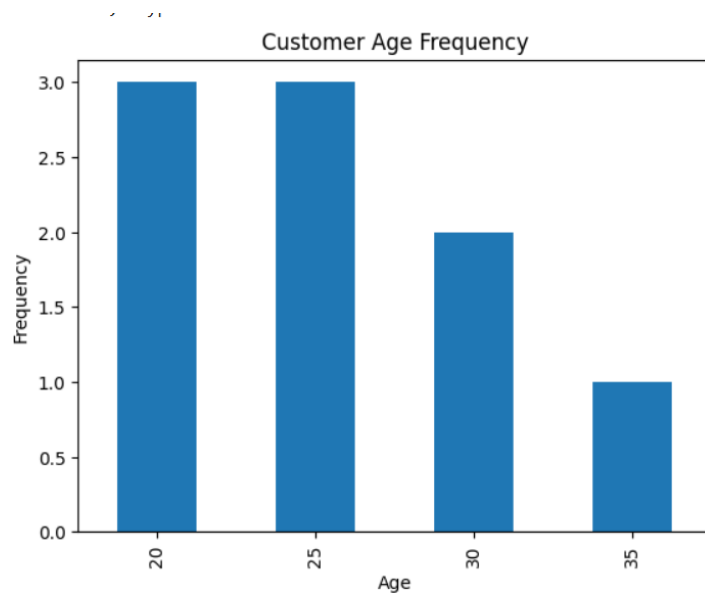
frequency = data["Age"].value_counts().sort_index()

print(frequency)

frequency.plot(kind="bar")
plt.title("Customer Age Frequency")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

OUTPUT:

```
Age
20    3
25    3
30    2
35    1
Name: count, dtype: int64
```



Question 15: Frequency Distribution of Likes

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.DataFrame({
    "Likes": [10, 20, 10, 30, 20, 10, 40, 30]
})

frequency = data["Likes"].value_counts().sort_index()

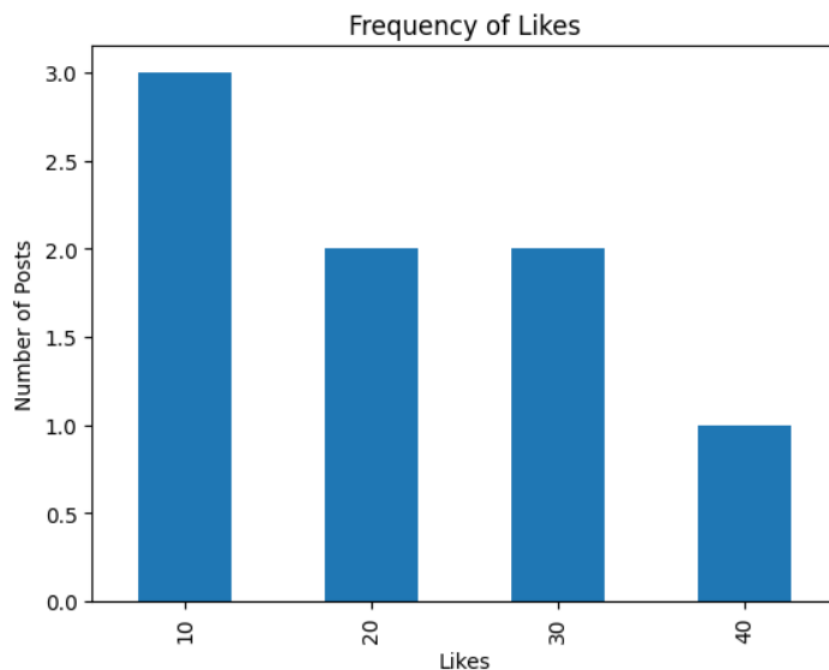
print(frequency)

frequency.plot(kind="bar")
plt.title("Frequency of Likes")
plt.xlabel("Likes")
plt.ylabel("Number of Posts")
plt.show()
```

OUTPUT:

```
Likes
10    3
20    2
30    2
40    1
Name: count, dtype: int64
```

Name: count, dtype: int64



Question 16: Word Frequency in Customer Reviews

```
import pandas as pd
import re
from collections import Counter

reviews = pd.DataFrame({
    "Review": [
        "good product and good service",
        "excellent product",
        "good quality product"
    ]
})

words = []

for review in reviews["Review"]:
    words.extend(re.findall(r"\b\w+\b", review.lower()))

frequency = Counter(words)

for word, count in frequency.most_common():
    print(word, ":", count)
```

OUTPUTS:

```
good : 3
product : 3
and : 1
service : 1
excellent : 1
quality : 1
```

Question 17: Customer Feedback – Top N Word Frequency

```
import pandas as pd

import re

from collections import Counter

import matplotlib.pyplot as plt


# Create sample data
data = pd.DataFrame({
    "feedback": [
        "The product is very good",
        "The product quality is good",
        "Good product and good service",
        "The service is excellent",
        "Product quality is excellent"
    ]
})

stop_words = {"the", "and", "is", "a", "an", "of", "to", "in"}

words = []

for text in data["feedback"].dropna():
    text = text.lower()
    words.extend(re.findall(r"\b[a-z]+\b", text))

words = [word for word in words if word not in stop_words]
```

```
frequency = Counter(words)
```

```
n = int(input("Enter N: "))
```

```
top_words = frequency.most_common(n)
```

```
print("\nTop Words:")
```

```
print(top_words)
```

```
names = [x[0] for x in top_words]
```

```
counts = [x[1] for x in top_words]
```

```
plt.bar(names, counts)
```

```
plt.xticks(rotation=45)
```

```
plt.title("Top Words in Customer Feedback")
```

```
plt.xlabel("Words")
```

```
plt.ylabel("Frequency")
```

```
plt.tight_layout()
```

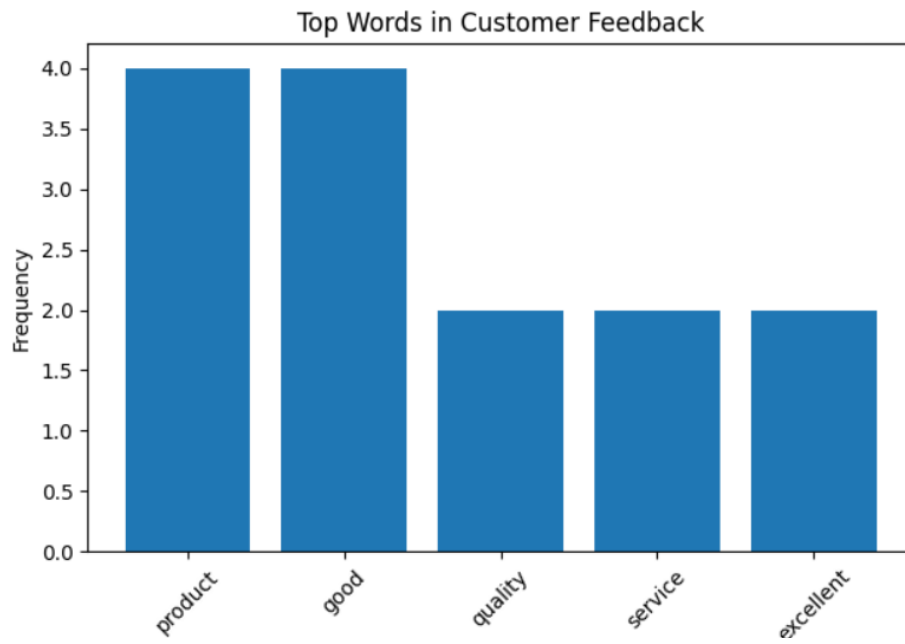
```
plt.show()
```

OUTPUT:

Enter N: 5

Top Words:

```
[('product', 4), ('good', 4), ('quality', 2), ('service', 2), ('excellent', 2)]
```



Question 18: Mean, Median, Standard Deviation, Boxplot, Scatter Plot and Q-Q Plot

```
import pandas as pd
import matplotlib.pyplot as plt
import scipy.stats as stats

data = pd.DataFrame({
    "Age": [20, 22, 25, 28, 30, 32, 35, 40],
    "Fat": [15, 18, 20, 22, 24, 27, 30, 32]
})

print("Mean:")
print(data.mean())

print("Median:")
print(data.median())

print("Standard deviation:")
print(data.std())

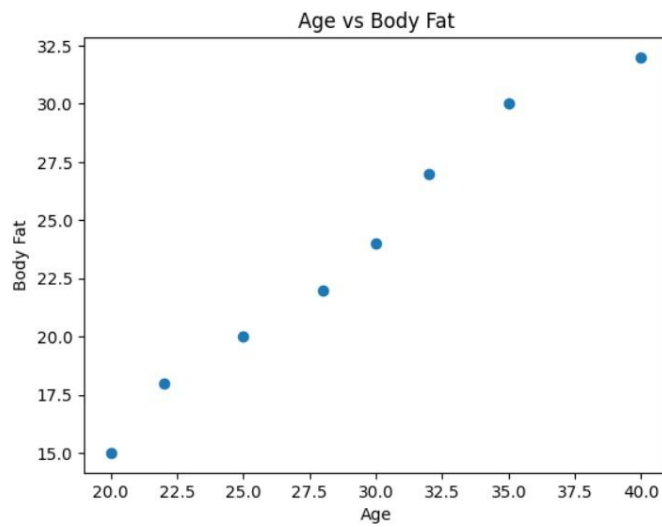
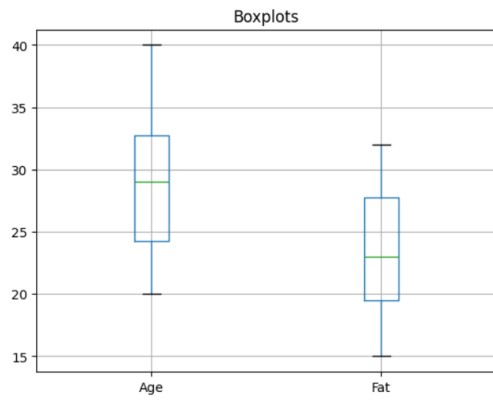
data[["Age", "Fat"]].boxplot()
plt.title("Boxplots")
plt.show()

plt.scatter(data["Age"], data["Fat"])
plt.xlabel("Age")
plt.ylabel("Body Fat")
plt.title("Age vs Body Fat")
plt.show()

stats.probplot(data["Age"], dist="norm", plot=plt)
plt.title("Q-Q Plot of Age")
plt.show()
```

OUTPUT:

```
Mean:
Age    29.0
Fat    23.5
dtype: float64
Median:
Age    29.0
Fat    23.0
dtype: float64
Standard deviation:
Age    6.697548
Fat    5.903994
dtype: float64
```



Question 19: 95% Confidence Interval for Blood Pressure Reduction

```
import numpy as np
from scipy import stats

drug = np.array([8, 10, 12, 9, 11, 13, 10, 12])
placebo = np.array([3, 5, 4, 6, 2, 5, 4, 3])

def confidence_interval(data):
    mean = np.mean(data)
    sem = stats.sem(data)
    interval = stats.t.interval(
        0.95, len(data) - 1, loc=mean, scale=sem
    )
    return interval

print("Drug 95% CI:", confidence_interval(drug))
print("Placebo 95% CI:", confidence_interval(placebo))
```

OUTPUT:

```
... Drug 95% CI: (np.float64(9.216289685290807), np.float64(12.033710314709193)
Placebo 95% CI: (np.float64(2.9053916697328868), np.float64(5.094608330267
```

Question 20: A/B Test – Compare Two Conversion Rate Samples

```
import numpy as np
from scipy.stats import ttest_ind

design_A = np.array([0.10, 0.12, 0.11, 0.13, 0.09, 0.12])
design_B = np.array([0.14, 0.15, 0.13, 0.16, 0.12, 0.14])

t_stat, p_value = ttest_ind(design_A, design_B)

print("t-statistic =", t_stat)
print("p-value =", p_value)

if p_value < 0.05:
    print("Statistically significant difference")
else:
    print("No statistically significant difference")
```

OUTPUT:

```
... t-statistic = -3.3999999999999995
    p-value = 0.006771072424337956
    Statistically significant difference
```
